



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and <u>Huffman coding</u> - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module I: Design Paradigms: Greedy, Divide and Conquer Techniques(6 Hours)

- **Overview and Importance of Algorithms**
 - **Stages of algorithm development:**
 - Describing the problem
 - Identifying a suitable technique
 - Design of an algorithm
 - Derive Time Complexity,
 - Proof of Correctness of the algorithm
- **Illustration of Design Stages**
 - **Greedy techniques:**
 - Fractional Knapsack Problem
 - **Huffman coding**
 - **Divide and Conquer:**
 - Maximum Subarray
 - Karatsuba faster integer multiplication algorithm.



Greedy Algorithm Strategy

- A greedy algorithm always makes the choice that looks best at the moment. i.e. it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.
- Most straightforward design technique
- These problems have n inputs and require us to obtain a subset that satisfies some constraints. Any subset that satisfies these constraints is called a **feasible solution**. We are required to find a feasible solution that either maximizes or minimizes a given objective function. A feasible solution that does this is called an **optimal solution**.



Huffman Codes and Data Compression

- It is a data compression technique- Also known as Variable length Coding.
- When the data transmitted from any source to destination – size of the data to compressed/reduced .
- Consider the following message: (For Example)

ABBCDBCCDAABBEEEEBEAB- 20 Characters



Simple Fundamentals

ABBCDBCCDAABBEEEEBEAB- 20 Characters

- For each and every character, What is the ASCII value?? From that convert in to Binary form
- For Example: “A”- 65 - 01000001- requires 8 Bits
- For Sending this message How many bits required??
- **$20 \times 8 = 160$ bits ----- Fixed length Coding**
- It is a process without Encoding and Decoding methods.



Simple Fundamentals

- Suppose we have 2 bits – how many possible combinations can be used for encoding a message?
- 2 bits- 4 Combinations – 00,01,10,& 11
- 3 bits- 8 Combinations- 000,001,010,011,100,101,110,111
- And so on.....
- Let us consider the message to be transmitted:
- **ABBCDBCCDAABBEEEEBEAB- 20 Characters**
- 5 characters are used- when we choose 3 bit encoding , can be able to accommodate 5 characters.



Simple Fundamentals

Value	Character
000	A
001	B
010	C
011	D
100	E
101	Remaining Unused
110	
111	

- **Encode**
 - **ABBCDBCCDAABBEEEEBEAB – $20 \times 3 = 60$ bits**
 - **000001001.....001**
- **Decode??**
 - **Send the table along with the message**
 - **It requires $5 \times 8 + 5 \times 3 = 55$ bits**
- **TOTAL: $60 + 55 = 115$ bits required**



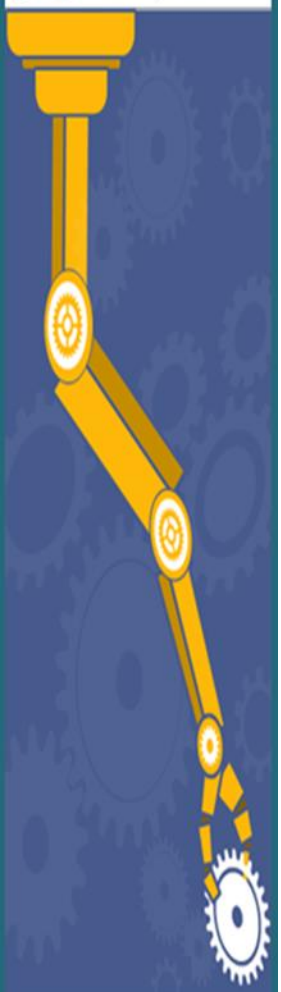
Huffman Codes- Variable length coding

- Huffman codes compress data very effectively, savings of 20% to 90% are typical, depending on the characteristics of the data being compressed.
- We won't use ASCII code here

Character	Frequency/Count
A	4
B	7
C	3
D	2
E	4

Huffman Codes- Variable length coding

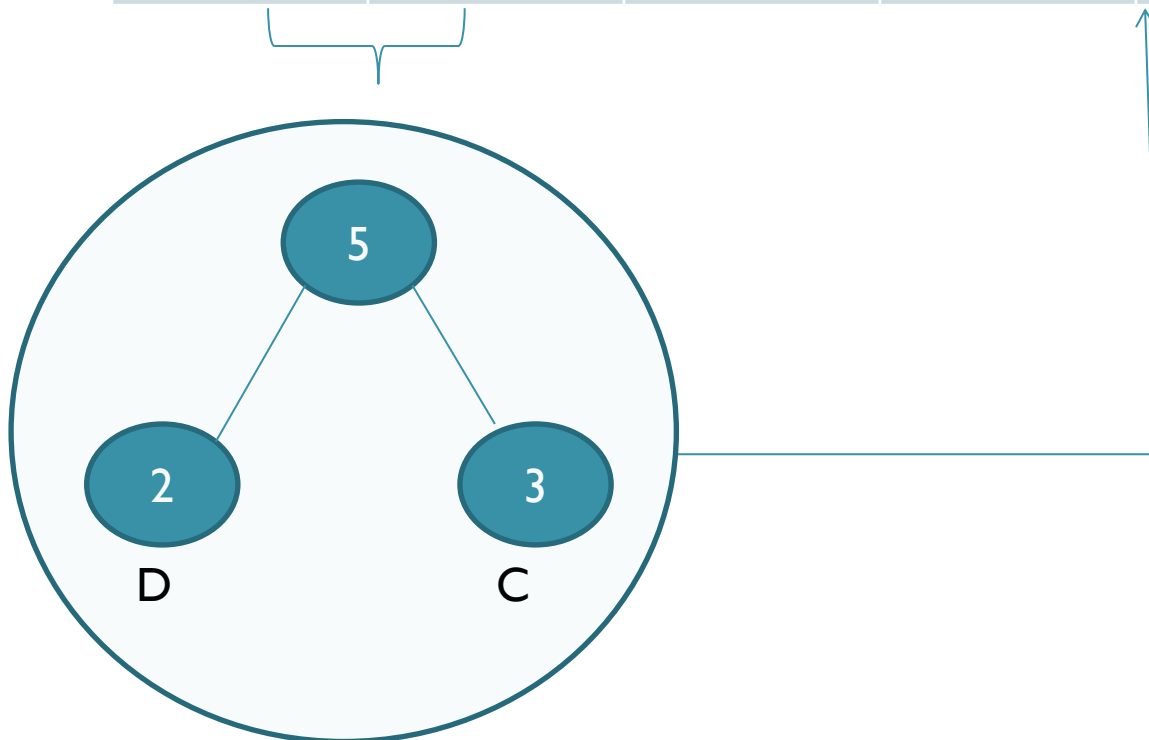
- STEP-1- Construction of Huffman Tree
- STEP-2- Traverse the Huffman Tree



STEP-1- Construction of Huffman Tree

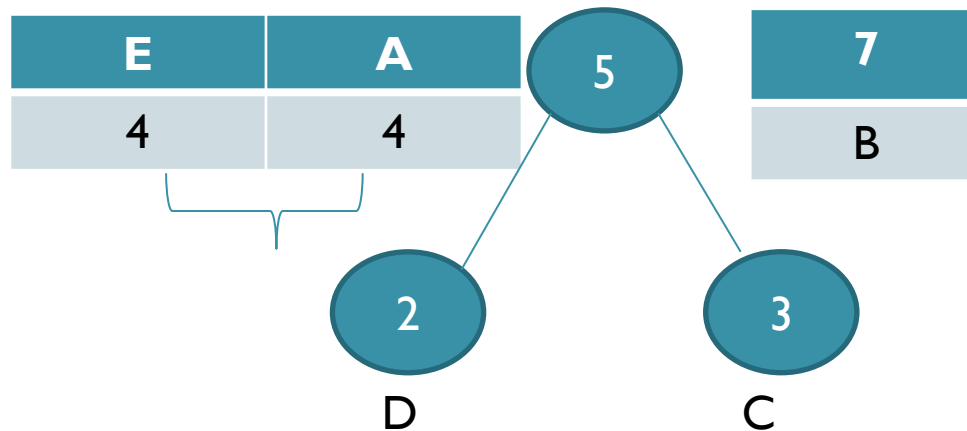
- Arrange the given characters in ascending order with respect to frequency/count

D	C	E	A	B
2	3	4	4	7



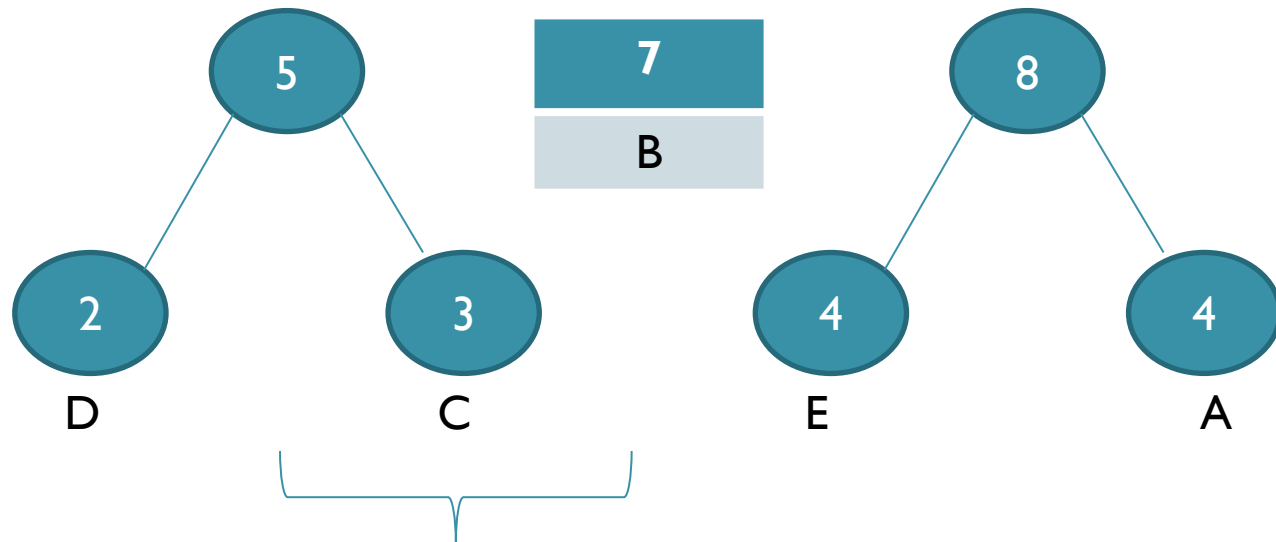
STEP-1- Construction of Huffman Tree

- Arrange the given characters in ascending order with respect to frequency/count

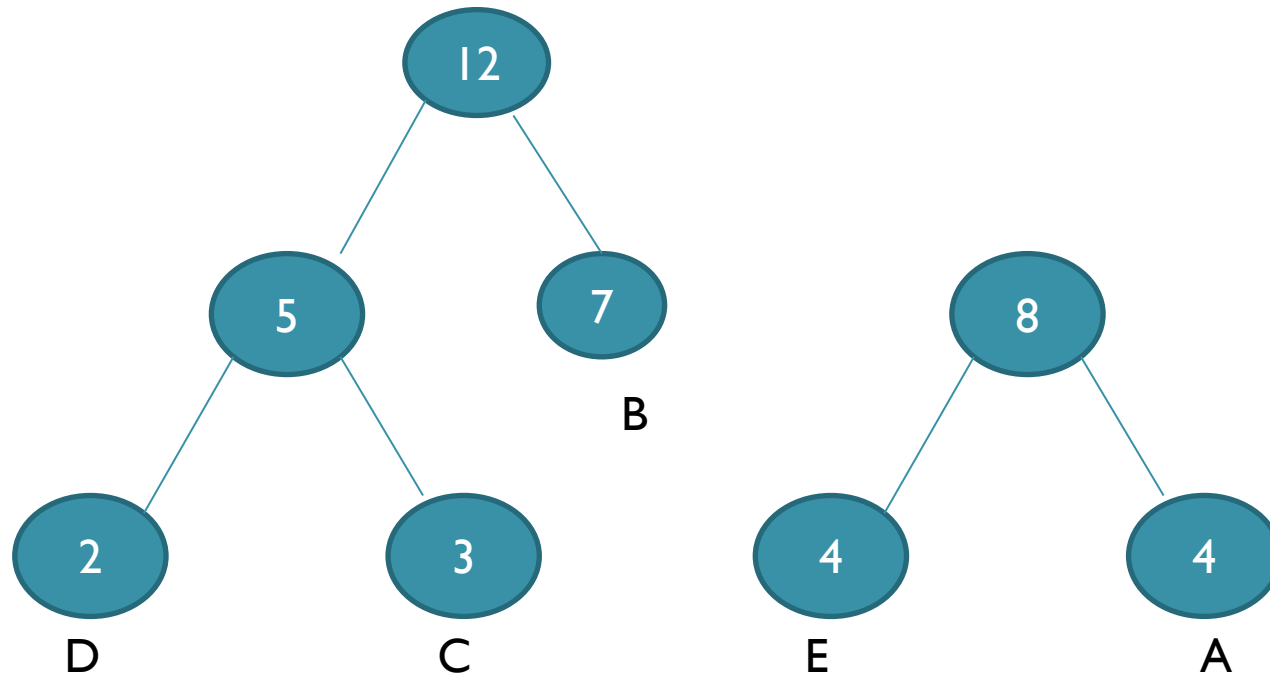


STEP-1- Construction of Huffman Tree

- Arrange the given characters in ascending order with respect to frequency/count



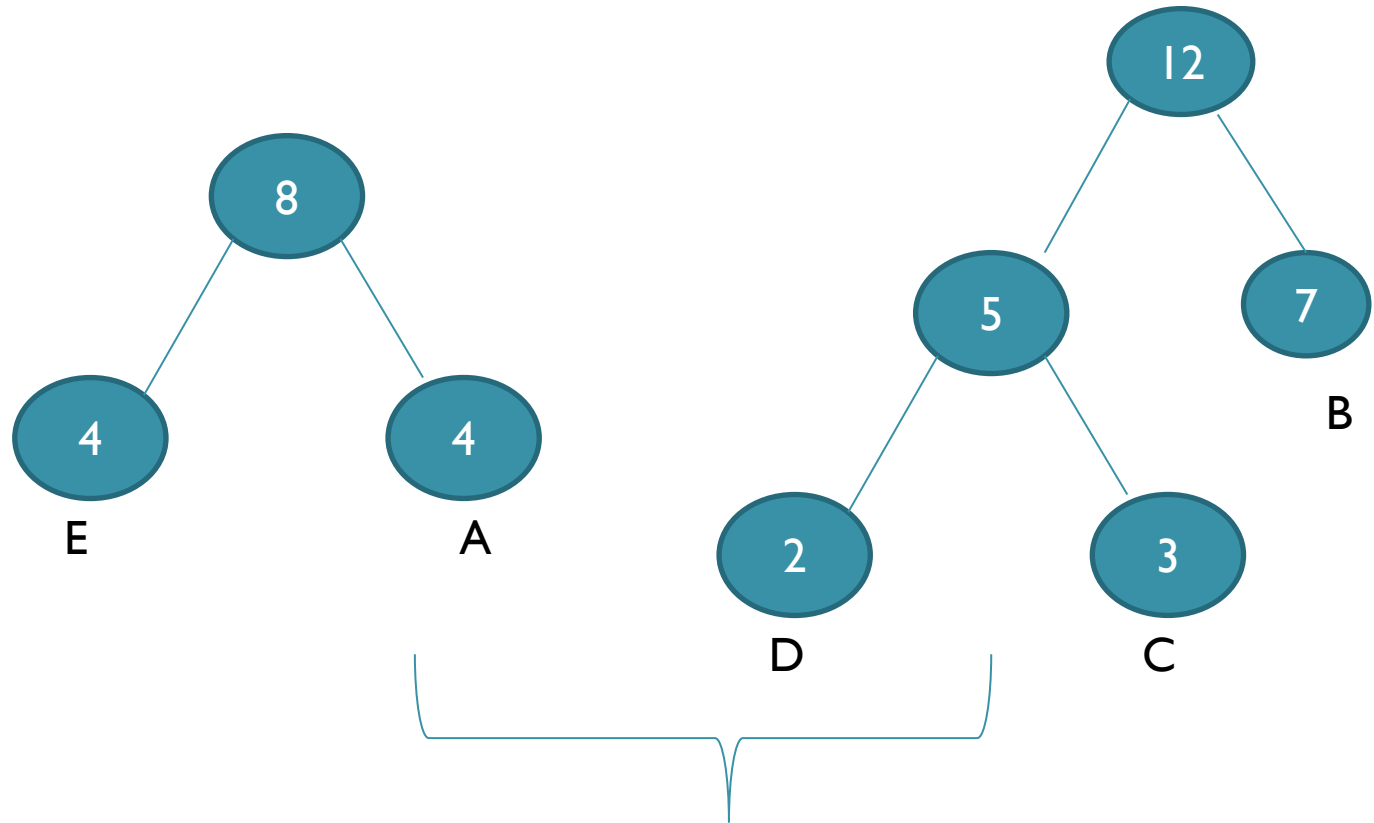
STEP-1- Construction of Huffman Tree



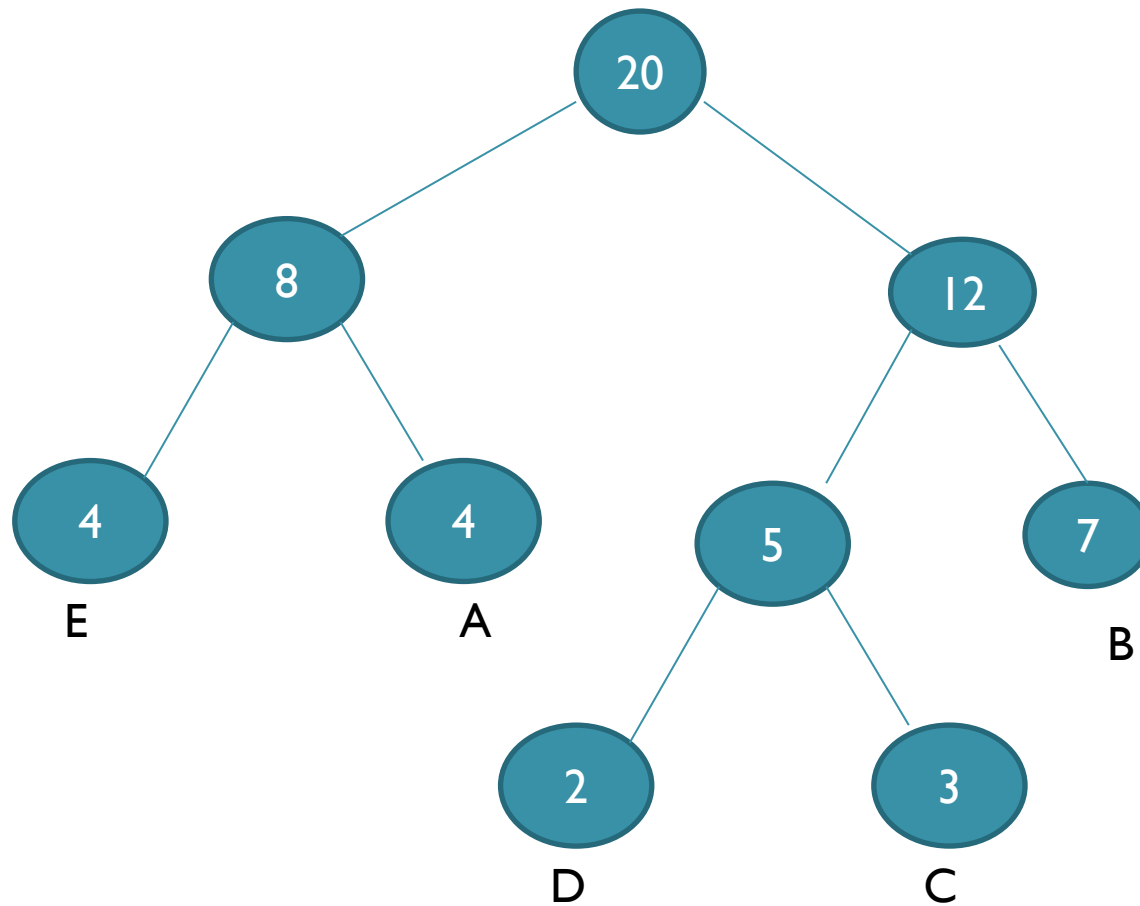
Arrange in sorted order



STEP-1- Construction of Huffman Tree



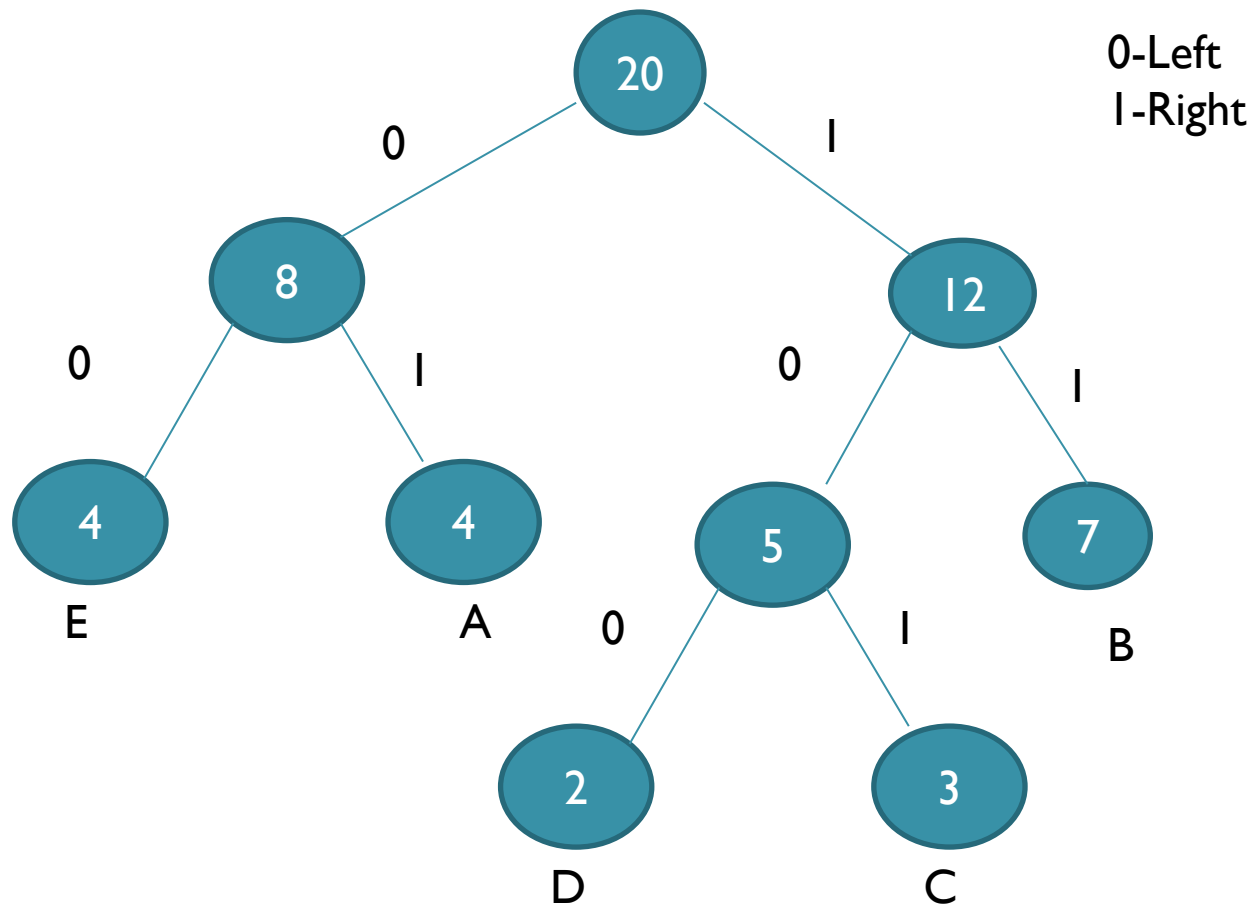
STEP-1- Construction of Huffman Tree



Final Tree



STEP-2- Traverse the Huffman Tree



Final Tree





Finding Huffman Code for each character

- Traverse the tree from the root node to reach the leaf nodes.

Character	Frequency/Count	Huffman Code
A	4	01
B	7	11
C	3	101
D	2	100
E	4	00

- ABBCDBCCDAABBEEEEBEAB- 20 Characters
- $4*2+7*2+3*3+2*3+4*2 = 45$ bits
- From table: $5*8=40+12=52$
- **TOTAL= 45+52=97 bits**

Huffman Codes and Data Compression

ENCODE **ABBCDBCCDAABBEEBEAB**

A	B	B	C	D	B	C	C	D	A	A	B	B	E	E	E	B	E	A	B
01	11	11	10 1	10 0	11	10 1	10 1	10 0	01	01	11	11	00	00	00	11	00	01	11

Next Question??- How the receiver decode the transmitted message??

011111011001110110110001011110000011000111

DECODE



Huffman Codes and Data Compression

- Huffman codes compress data very effectively, savings of 20% to 90% are typical, depending on the characteristics of the data being compressed.
- Let us consider the Data to be sequence of characters.
- Huffman's greedy algorithm uses a table giving how often each character occurs (i.e., its frequency) to build up an optimal way of representing each character as a binary string.



Huffman Codes and Data Compression

- Suppose we have a 100,000-character data file that we wish to store compactly. We observe that the characters in the file occur with the frequencies given in the figure. That is, only 6 different characters appear, and the character “a” occurs 45,000 times.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100



Huffman Codes and Data Compression

- Binary Character Code

- If we use a fixed-length code, we need 3 bits to represent 6 characters: a = 000, b = 001, ..., f = 101. This method requires 300,000 bits to code the entire file. Can we do better?
- A variable-length code can do considerably better than a fixed-length code, by giving frequent characters short code words and infrequent characters long code- words.
- Here the 1-bit string 0 represents “a”, and the 4-bit string 1100 represents “f”. This code requires

$(45.1 + 13.3 + 12.3 + 16.3 + 9.4 + 5.4) \cdot 1,000 = 224,000$ bits to represent the file, **a savings of approximately 25%.**



Solve

CHAR	A	B	C	D	-
PROBABILITY	0.35	0.1	0.2	0.2	0.1

1. Average length of the message
2. ENCODE DAD
3. DECODE 10011011011101



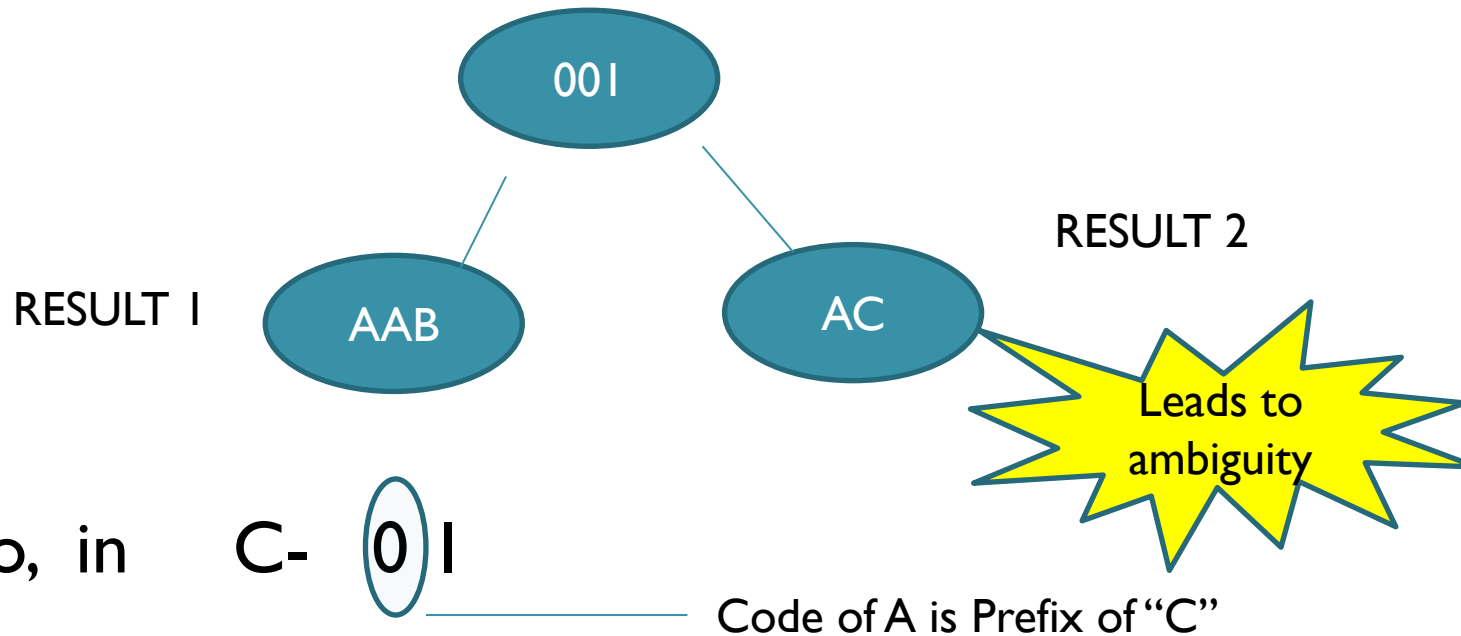
Huffman Codes and Data Compression

- Prefix Code
 - Huffman coding works on the technique called Prefix Code
 - No Code is prefix of another code
 - Let us take the example: A-0, B-1, C-01
 - **Message sent is 001**
 - **What happens when it is decoded??**



Huffman Codes and Data Compression

- What happens when it is decoded??



This should not happen....



Huffman Codes - Algorithm

```

Huffman (C) // C- Set of n characters {a,b,c,d,e}
{
    // f(C) = {50,25,15,40,75}

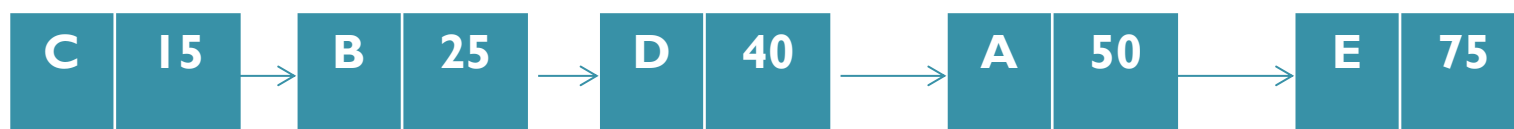
    n=|C| // 5
    Q ← C // Min- Priority Queue
    for i=1 to n-1
    {
        z= allocate-Node () // create a new node
        x= left[z]=Extract-Min(Q) // Use Min heap
        y= right[z] =Extract-Min(Q) // Use Min heap
        f [z]=f[x]+f[y]
        Insert (Q, z)// Inserting node in linked list
    }
    return Extract-Min (Q)// Returns the root of the tree
}
    
```



Find an optimal Huffman Code for the following set of frequencies

CHAR	A	B	C	D	E
Frequency	50	25	15	40	75

- Given: $C = \{a, b, c, d, e\}$, $f(C) = \{50, 25, 15, 40, 75\}$, $n=5$
- Create min- Priority queue



Priority Queues - Operations

- Insertion of elements in the priority queue is similar to the **ordinary queue**
- Deletion of the elements is done according to the **priority**
- The elements in the priority queue have some priority. The priority of the element is used to determine the order in which the elements will be processed.



Node Structure- Priority queue

Struct Node

{

int priority;

int info;

struct Node *link;

};

PRIORITY

INFO

LINK

NODE

Iteration process- i=1



$x = \text{left}[z]$



$y = \text{right}[z]$

$z = \text{allocate-Node}()$

$x = \text{left}[z] = \text{Extract-Min}(Q)$

$y = \text{right}[z] = \text{Extract-Min}(Q)$

$f[z] = f[x] + f[y] = 40$

$\text{Insert}(Q, z)$

Iteration process- i=2

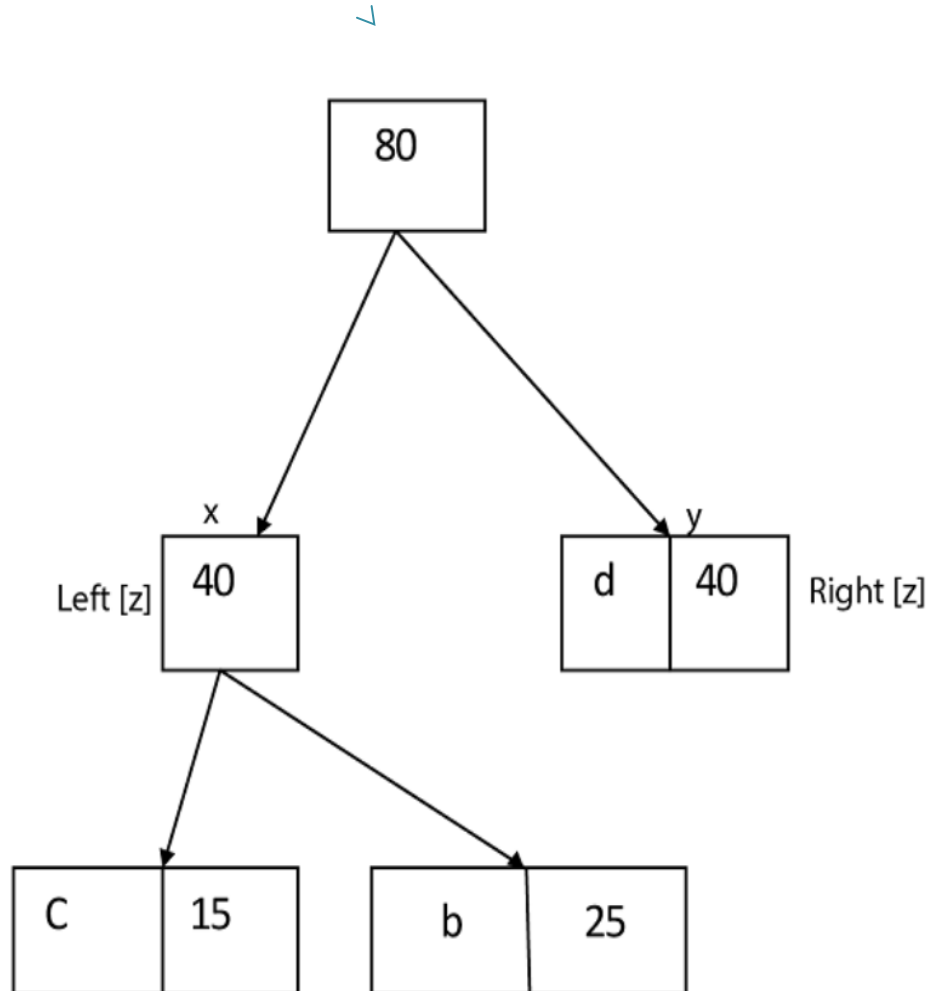
z= allocate-Node ()

x= left[z]=Extract-Min(Q) // 40

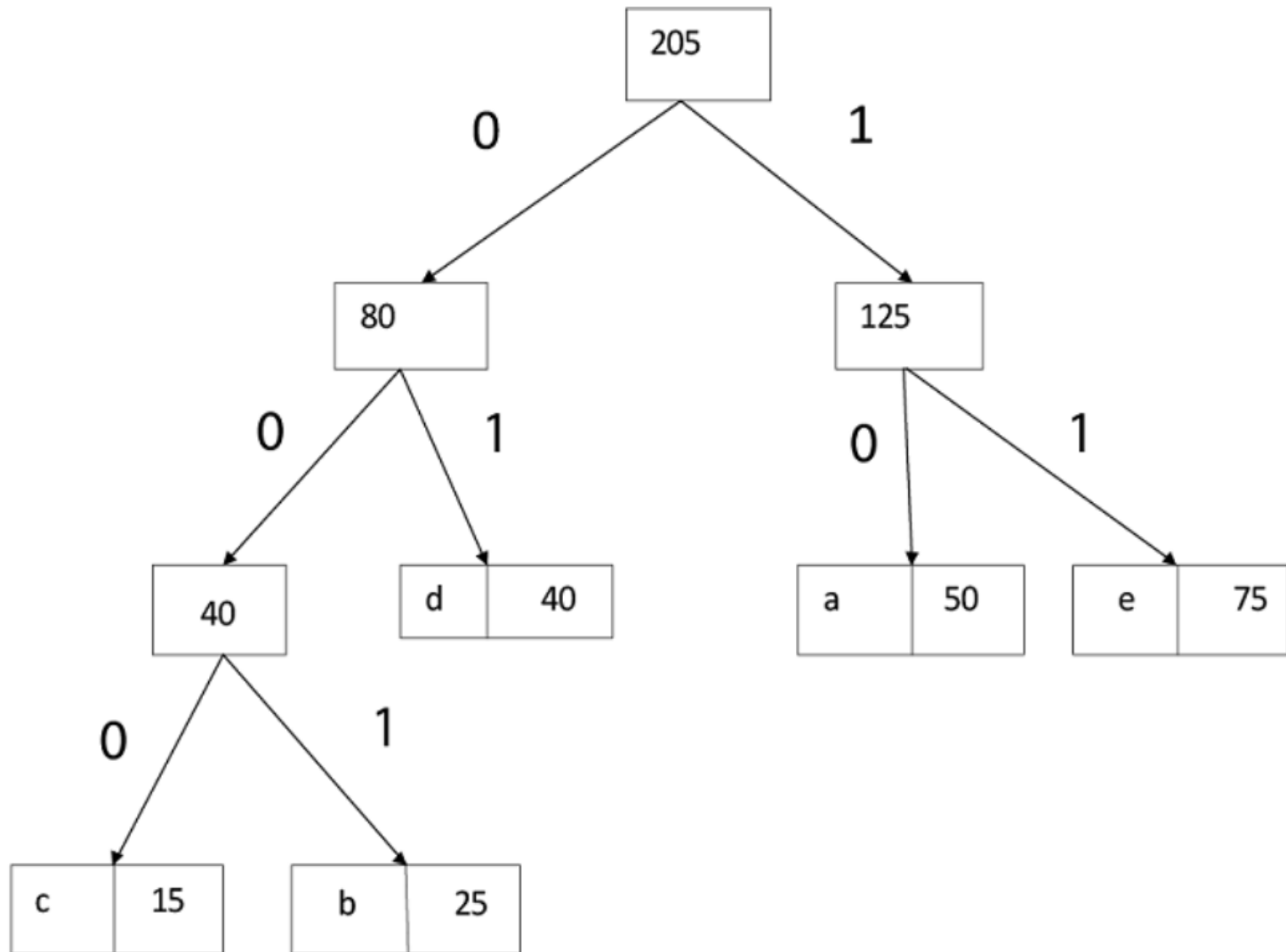
y= right[z] =Extract-Min(Q)//40

f [z]=f[x]+f[y] = 80

Insert (Q, z)



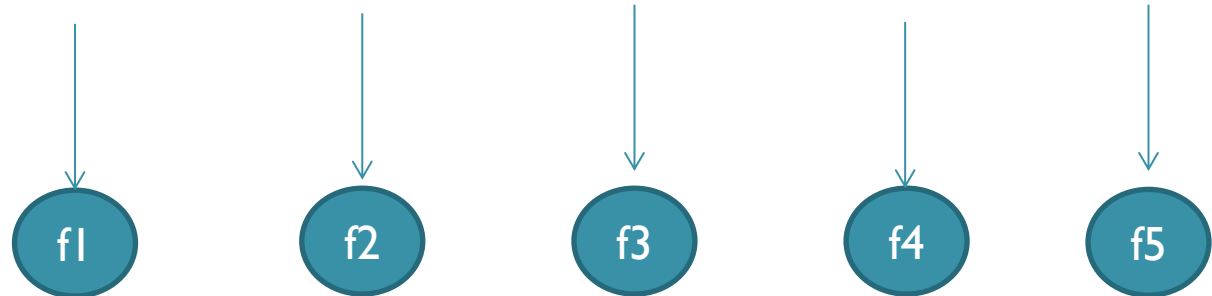
Final Output



Huffman Codes - Analysis

N CHARACTERS/KEYS

A	B	C	D	E
f1	f2	f3	f4	f5



N individual trees/nodes

Maintain the trees initially.

What is our first requirement??

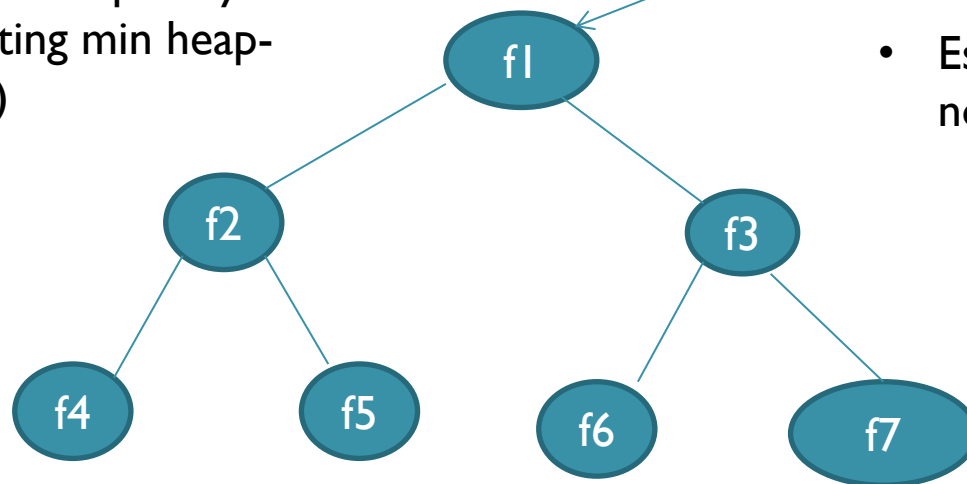
Select first two minimum and merge- What is the procedure adopted here??

MIN heap



Huffman Codes - Analysis

Time complexity of
creating min heap-
 $O(n)$



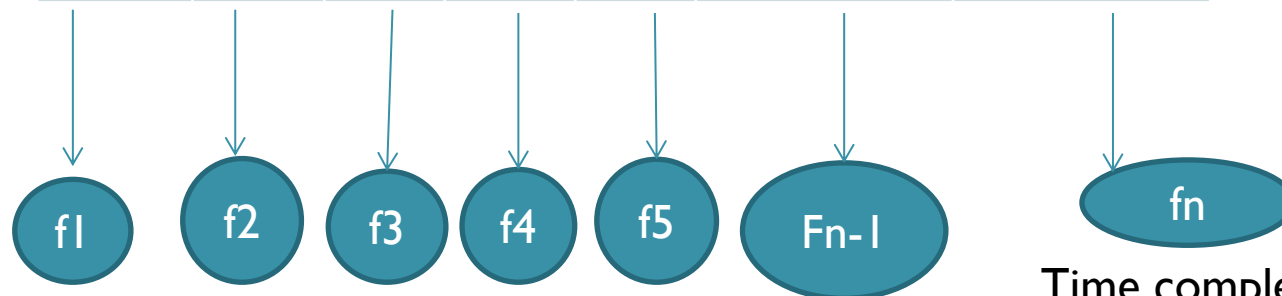
Min heap tree

- Establish connection between node and individual tree

Individual tree

N characters

a	b	b	d	e	f	g
f1	f2	f3	f4	f5	f6	f7



N trees

Time complexity of
creating n trees- $O(n)$

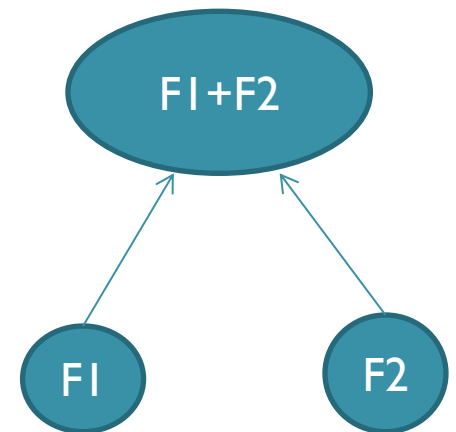
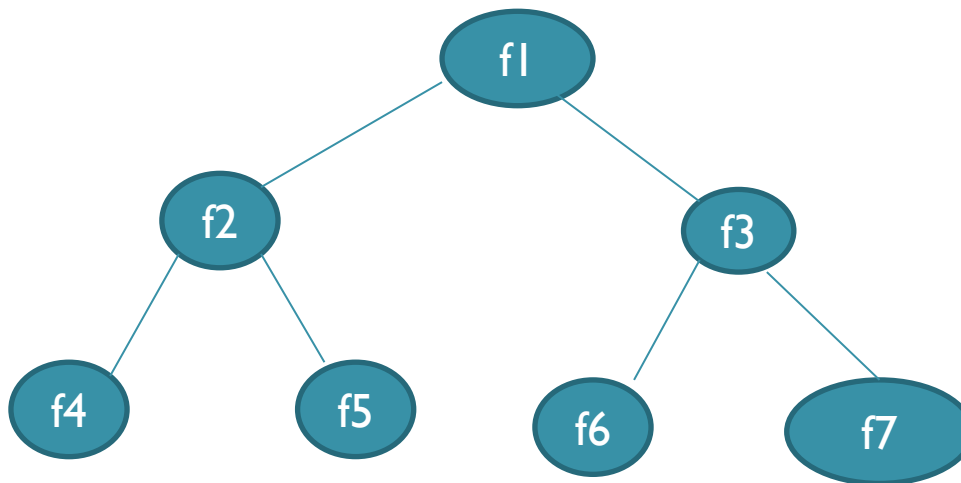
Huffman Codes – Algorithm Requirements

- Basically Algorithm executes in two steps:
 - **Step 1: Taking top two frequency elements**
 - How to take top two?
 - Consider the min heap and pick the element and rearrange(ADJUST) the tree and pick the next element.
 - What is the time Complexity of doing this?
 - $\log n$ (Time complexity of PICKING a node from the tree is $\log n$ because height of the tree is $\log n$) **So $\log n + \log n = 2\log n$**



Huffman Codes – Algorithm Requirements

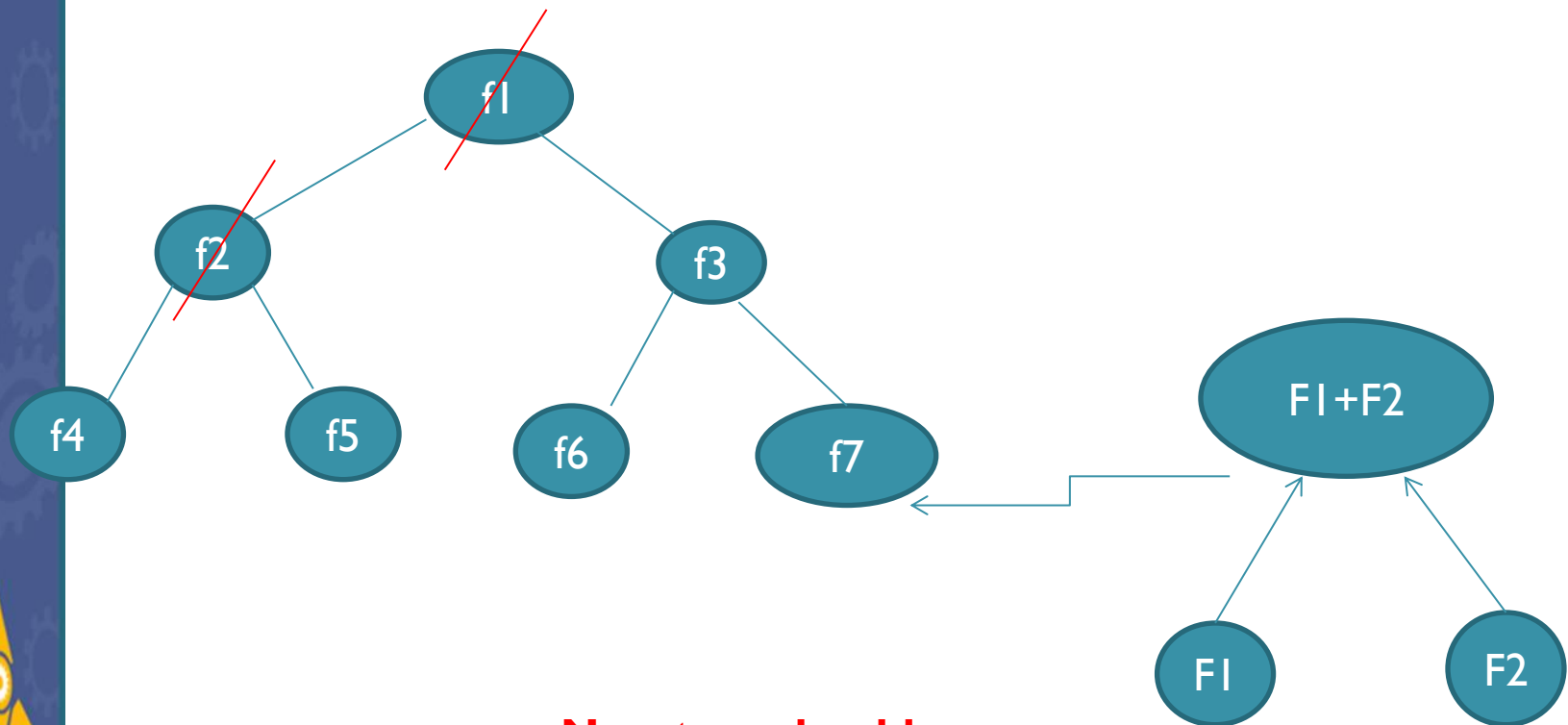
- Basically Algorithm executes in two steps:
 - **Step 2: MERGING** top two frequency elements



What is the time complexity of merging? – constant time



Huffman Codes – Algorithm Requirements



- **New tree should be updated in the existing tree**
- **Adjust the tree using min heap property**



Huffman Codes - Analysis

- Selecting Top 2 min frequency takes $(\log n + \log n) = 2 \log n$ (*Note: Height of the tree is $\log n$*)
- Merge operation - constant time
- Adjusting the merged node in the binary heap - $\log n$
- The above operation repeated for $n-1$ times
- $(n-1)(3\log n + \text{constant time})$
- $\approx O(n \log n)$



Huffman Codes - Analysis

- To analyse the running time of Huffman's algorithm, we assume that Q is implemented as a binary min-heap .
- For a set C of n characters, we can initialize Q in $O(n)$ time using the **M IN -H EAP** algorithm
- The for loop executes exactly $n - 1$ times, and since each heap operation requires time $O(\log n)$, the loop contributes $O(n \cdot \log n)$ to the running time.
- **$O(n \cdot \log n)$**

